

Project Documentation – EduGenie-AI

Google Gemini Powered AI Learning Assistant

Page 1 – Documentation Overview and Project Introduction

Project Documentation provides a complete written record of EduGenie-AI, including its purpose, requirements, design, implementation, testing, and usage. It helps students, teachers, evaluators, and future developers understand how the system was developed.

Project Overview

EduGenie-AI is a Google Gemini powered AI learning assistant designed to support students with common learning activities. The application provides question answering, topic explanations, quiz generation, summarization, and learning recommendations through a web interface.

Purpose of Documentation

- Explain the problem addressed by the project.
- Describe the system requirements and architecture.
- Document technologies, modules, APIs, and configuration.
- Record testing methods and expected behavior.
- Provide instructions for running and using the application.
- Support project presentation, evaluation, maintenance, and future enhancement.

Technology Stack

Layer	Technology
Programming	Python
Backend	FastAPI + Uvicorn
AI	Google Gemini + Google GenAI SDK
Frontend	HTML, CSS, JavaScript
Development	Visual Studio Code
Version Control	Git / GitHub

Project Documentation – EduGenie-AI

Google Gemini Powered AI Learning Assistant

Page 2 – System Requirements and Architecture Documentation

The documentation records both functional and non-functional requirements so that the implemented system can be compared with the original project objectives.

Functional Requirements

- The system shall accept student questions and generate answers.
- The system shall explain academic topics in simple language.
- The system shall generate quiz-oriented content.
- The system shall summarize supplied learning material.
- The system shall provide learning recommendations.
- The system shall display results through a web interface.

Non-Functional Requirements

- The interface should be simple and readable.
- The backend should provide reliable API routes.
- API credentials must remain private.
- The application should handle common errors gracefully.
- The modular structure should support maintenance and future development.

Architecture Documentation

Component	Responsibility
Frontend	Collect input and display results
FastAPI	Manage routes and application flow
Learning Modules	Perform individual educational tasks
Gemini	Generate AI-based learning content
Environment Configuration	Store API key and model settings securely

The architecture follows a clear request flow: student input → frontend → FastAPI route → learning module → Gemini → backend response → frontend display.

Project Documentation – EduGenie-AI

Google Gemini Powered AI Learning Assistant

Page 3 – Implementation and Module Documentation

The implementation is organized into separate files and modules. This structure makes the application easier to understand, test, debug, and extend.

Major Project Files

File / Area	Purpose
main.py	FastAPI application and API routes
qna.py	Question answering using Gemini
explanation_module.py	Topic explanation functionality
quiz_module.py	Quiz generation
summary_module.py	Text summarization
learning_path.py	Learning recommendations
templates/index.html	Main web page
static/css/style.css	Interface styling
static/css/js/app.js	Frontend API requests and result handling
.env	Private Gemini configuration

Implementation Process

- Set up Python virtual environment and dependencies.
- Configure Gemini credentials through environment variables.
- Develop and test individual learning modules.
- Create FastAPI routes for each feature.
- Develop the HTML, CSS, and JavaScript interface.
- Connect frontend requests to backend routes.
- Test the complete application workflow.

Security Documentation

The Gemini API key is kept in the .env configuration and excluded from Git through .gitignore. The key should never be placed directly in frontend files or shared in project documentation.

Project Documentation – EduGenie-AI

Google Gemini Powered AI Learning Assistant

Page 4 – Testing, Usage and Troubleshooting Documentation

Testing documentation records how EduGenie-AI is verified before demonstration and submission.

Testing Areas

- Home page and health endpoint testing.
- Question answering and explanation testing.
- Quiz and summary testing.
- Learning recommendation testing.
- Frontend-backend integration testing.
- Empty and invalid input testing.
- Gemini configuration and error testing.

Basic Usage Procedure

- Start the Python virtual environment.
- Ensure the .env file contains the required Gemini configuration.
- Run the FastAPI application with Uvicorn.
- Open the application in a web browser.
- Enter a question or select a learning feature.
- Submit the request and review the generated result.

Common Troubleshooting

Problem	Suggested Check
Server fails to start	Check Python environment and installed packages
AI request fails	Check Gemini API key and model configuration
Page does not load	Check Uvicorn status and browser URL
Frontend request fails	Check JavaScript endpoint and FastAPI route
Unexpected result	Review prompt and input

Troubleshooting should begin with the exact terminal or browser error message. After a fix is applied, the affected test should be repeated.

Project Documentation – EduGenie-AI

Google Gemini Powered AI Learning Assistant

Page 5 – Documentation Deliverables and Future Scope

The final documentation package should contain enough information for another person to understand, run, test, and demonstrate EduGenie-AI.

Recommended Documentation Contents

- Project abstract and introduction.
- Problem statement and objectives.
- Requirement analysis.
- System architecture and project design.
- Implementation details and module descriptions.
- Testing strategy and test results.
- Screenshots of the application.
- Installation and usage instructions.
- Limitations and future enhancements.
- Conclusion and references.

Future Scope

- Student login and personalized profiles.
- Database storage for learning history.
- Quiz scoring and progress tracking.
- Multilingual learning support.
- Voice input and voice responses.
- Teacher dashboards and analytics.
- More advanced personalized learning paths.

Conclusion

Project Documentation forms the permanent technical and academic record of EduGenie-AI. It explains what the project does, how it was designed and implemented, how it was tested, and how it can be improved. Good documentation also makes the final demonstration easier to organize and explain.

Project: EduGenie-AI | Technology: FastAPI + Google Gemini + HTML/CSS/JavaScript